Post-Mortem del proyecto Teknical May Cry (TMC)

Proyecto Integrador Anual (P.I.A.) — 2026

E.E.S.T. N.º 1 “Eduardo Ader” — Vicente López

5.º año, 3.ª división

Laboratorio de Programación (LPR)

Integrantes: Del Pino, Mattia, Orue y Ferrari

Docentes: Ganduglia, Yamil, Mansilla, Muñoz, York y Elías

11 de noviembre de 2026

Post-Mortem del proyecto Teknical May Cry (TMC)

1. Introducción

El presente documento tiene como finalidad realizar un análisis retrospectivo del desarrollo del proyecto Teknical May Cry (TMC), identificando los principales logros, las dificultades encontradas, las decisiones técnicas y los aprendizajes obtenidos durante su realización.

El post-mortem permite evaluar tanto los resultados técnicos como los aspectos organizativos del equipo. Su propósito es reconocer los aciertos, analizar los problemas y detectar oportunidades de mejora que puedan aplicarse en futuras etapas o proyectos.

Este análisis contempla las distintas áreas involucradas en el desarrollo, principalmente el software, el diseño del videojuego, la plataforma web, la organización del repositorio y la documentación técnica.

2. Descripción general del proyecto

TMC es un proyecto desarrollado en el marco del Proyecto Integrador Anual 2026. Consiste en una propuesta de videojuego inspirada en Pokémon Rojo Fuego, que incorpora diferentes mecánicas y minijuegos para ampliar la experiencia de juego.

Entre las propuestas trabajadas se encuentra un minijuego inspirado en Five Nights at Freddy’s. Además del videojuego, el proyecto contempla una plataforma web complementaria destinada a presentar y centralizar información y recursos relacionados con el sistema.

El proyecto se aborda como una experiencia integral de desarrollo, en la que se combinan programación, diseño, organización de archivos, documentación y trabajo colaborativo.

3. Objetivos iniciales

Los objetivos generales planteados para el proyecto incluyen:

- Desarrollar un videojuego con mecánicas inspiradas en títulos clásicos.
- Incorporar minijuegos que aporten variedad a la experiencia de juego.
- Diseñar una interfaz visual coherente con la identidad del proyecto.
- Desarrollar una plataforma web complementaria.
- Organizar el código fuente y los recursos mediante un repositorio de GitHub.
- Aplicar conocimientos de programación, diseño y desarrollo de sistemas.
- Documentar el proceso de desarrollo y las decisiones técnicas.

4. Resultados obtenidos

Durante el desarrollo se trabajó sobre diferentes componentes del sistema. Esta sección debe completarse con el estado real de cada funcionalidad al momento de presentar la versión definitiva del documento.

4.1. Desarrollo del software

Se trabajó en la estructura del videojuego, sus recursos gráficos, las mecánicas principales y la incorporación de minijuegos. Para la versión final del post-mortem, se recomienda enumerar cuáles de estas funcionalidades quedaron implementadas, cuáles se encuentran en desarrollo y cuáles no pudieron completarse.

4.2. Desarrollo web

Se desarrolló una plataforma web complementaria para presentar el proyecto y centralizar sus recursos. En la versión definitiva se deberán especificar las secciones y funcionalidades que efectivamente se encuentran operativas.

4.3. Organización del repositorio

Se utilizó GitHub como herramienta de almacenamiento y organización del código fuente. La separación de archivos y documentación en directorios permite mantener los recursos centralizados y facilita su consulta por parte de los integrantes y evaluadores.

4.4. Documentación

Se estableció una estructura documental para registrar los aspectos técnicos, el desarrollo y las evidencias del proyecto. La documentación deberá actualizarse para reflejar el estado real del software y del hardware.

5. Aspectos positivos

Entre los aspectos positivos que pueden destacarse del proceso se encuentran la integración de conocimientos adquiridos en distintas áreas, la posibilidad de desarrollar diferentes funcionalidades dentro de un mismo proyecto y la utilización de herramientas digitales para organizar el trabajo.

La incorporación de minijuegos permitió explorar distintas mecánicas y ampliar el alcance de la propuesta. Asimismo, el uso de un repositorio compartido ofrece una base para ordenar los archivos y registrar las modificaciones realizadas.

Estos puntos deberán revisarse grupalmente para incorporar ejemplos concretos que demuestren qué decisiones o prácticas resultaron especialmente útiles.

6. Dificultades encontradas

Como ocurre habitualmente en proyectos de desarrollo tecnológico, pueden surgir inconvenientes que afectan los tiempos y la implementación. Para que este apartado represente fielmente la experiencia del equipo, es necesario conservar únicamente los problemas que efectivamente hayan ocurrido.

- Integración de funcionalidades: compatibilizar diferentes mecánicas dentro de un mismo videojuego.
- Organización del código: mantener una estructura clara y evitar conflictos entre archivos.
- Desarrollo web: integrar correctamente los distintos componentes de la plataforma.
- Gestión del tiempo: distribuir las tareas de acuerdo con los plazos académicos.
- Coordinación grupal: mantener una comunicación constante y una distribución equilibrada de responsabilidades.

7. Soluciones y decisiones implementadas

Las decisiones adoptadas durante el desarrollo deben describirse vinculándolas con problemas concretos. A modo de estructura inicial, se pueden documentar la centralización del código fuente en GitHub, la separación de recursos según su función y la organización de la documentación en apartados independientes.

También resulta conveniente registrar las decisiones que hayan permitido corregir errores, simplificar procesos o mejorar la colaboración. En la versión final se deberán agregar ejemplos específicos y evitar incluir soluciones que no hayan sido utilizadas.

8. Aspectos que podrían mejorarse

A partir del análisis del proceso, se proponen las siguientes oportunidades de mejora para futuros desarrollos:

- Planificación: establecer objetivos parciales y fechas de entrega específicas.
- Documentación: registrar las decisiones técnicas a medida que avanza el desarrollo.
- Pruebas: verificar cada funcionalidad antes de integrarla con el resto del sistema.
- Organización: definir responsabilidades y tareas concretas para cada integrante.

- Control de versiones: mantener criterios uniformes para registrar modificaciones en GitHub.
- Integración: planificar desde las primeras etapas la relación entre los diferentes componentes del sistema.

9. Lecciones aprendidas

El desarrollo de TMC permite reconocer que la creación de un sistema informático no depende únicamente de escribir código. También requiere planificación, organización, investigación, comunicación y capacidad para resolver problemas.

Entre los aprendizajes que el equipo puede evaluar se encuentran la importancia de diseñar una estructura organizada, probar las funcionalidades durante el proceso, utilizar herramientas de control de versiones, mantener la documentación actualizada y distribuir las responsabilidades de manera clara.

Para fortalecer esta sección, se recomienda agregar situaciones concretas en las que el equipo haya aplicado estos aprendizajes.

10. Conclusión

El desarrollo de Teknical May Cry constituye una experiencia de integración de conocimientos técnicos mediante la creación de un proyecto que combina diferentes áreas del desarrollo informático.

El análisis retrospectivo permite reconocer los avances alcanzados, identificar las dificultades que deben resolverse y establecer criterios para mejorar futuras etapas de trabajo. La evaluación final deberá basarse en los resultados efectivamente obtenidos y en la experiencia real de todos los integrantes.

El post-mortem no debe entenderse únicamente como una evaluación de los errores, sino como una herramienta para transformar la experiencia adquirida en conocimientos aplicables a futuros desarrollos.

11. Información pendiente de completar

Antes de entregar la versión definitiva, el equipo deberá revisar y completar los siguientes puntos:

- Indicar qué funcionalidades del videojuego quedaron terminadas, parcialmente terminadas o pendientes.
- Describir los problemas técnicos reales y las soluciones que se aplicaron.
- Incorporar una evaluación concreta de la organización y distribución de tareas.
- Agregar el estado real del área de hardware, sin atribuirle componentes o resultados no confirmados.
- Incluir evidencias, capturas o referencias al repositorio cuando corresponda.